

Master Flightline Clock

Recommended Software Architecture and Development Specification

This document captures the recommended software architecture, development environment, project organisation and engineering practices for the Master Flightline Clock project. The intention is to create a maintainable, reliable and expandable system suitable for real-world competition use.

Project Philosophy

The system should not be viewed simply as a clock. It is a deterministic real-time control system that manages race timing, operator interaction and public display output. The clock display is only one component of the overall system.

Primary Components

Component	Purpose
Clock Engine	Controls system state, timing and business logic.
Operator Interface	Allows configuration of the next heat and system management.
Display Renderer	Produces the full-screen output shown on the LED matrix display.
Configuration & Storage	Stores settings, defaults, history and competition data.

Technology Stack

- Language: Python 3.13
- GUI: PySide6 (Qt)
- Database: SQLite
- Configuration: JSON files
- IDE: Visual Studio Code
- Version Control: Git with a private GitHub repository

Recommended Python Packages

- PySide6
- SQLAlchemy

- Pydantic
- pytest
- Black
- Ruff
- GitLens (VS Code extension)
- Pylance (VS Code extension)

Project Directory Structure

```
MasterFlightlineClock/  
|  
├─ README.md  
├─ requirements.txt  
├─ .gitignore  
├─ config/  
├─ docs/  
├─ assets/  
|   ├── fonts/  
|   └─ logos/  
├─ database/  
|   └─ flightline.db  
├─ src/  
|   ├── main.py  
|   ├── clock_engine.py  
|   ├── heat_manager.py  
|   ├── timer_engine.py  
|   ├── display_window.py  
|   ├── ui_window.py  
|   ├── settings.py  
|   ├── models.py  
|   ├── database.py  
|   ├── events.py  
|   └─ colours.py  
├─ tests/  
└─ logs/
```

Display Architecture

The HDMI-to-HUB75 hardware should be treated as a standard second monitor. The display renderer opens a borderless full-screen window on the secondary display, allowing Windows to manage the output while the LED controller simply reproduces the image.

State Machine Design

The application should be built around a formal state machine rather than scattered conditional logic.

State	Description
Idle	Application started. No heat configured.
Configured	Operator entering next heat.
Ready	Valid heat data entered. Waiting for activation.
Running	Heat currently active.
Finished	Heat completed automatically.
Cancelled	Heat cancelled by operator.

Button enablement should be driven entirely by the current application state. For example, the Activate button is only enabled in the Ready state, while the Cancel button is only enabled in the Running state.

Heat Model

The software should maintain two independent data structures:

- **Active Heat** - the heat currently displayed and running.
- **Next Heat** - the heat being edited by the operator.

When the operator presses Activate, the Next Heat is copied into Active Heat and timing commences. The operator is immediately free to configure the subsequent heat.

Configuration Files

Configuration should be stored as JSON files, including display colours, fonts and default timings. Business logic should not contain hard-coded values wherever practical.

Database

SQLite is recommended for storing competition history, heat records, operator settings and future event replay capability. As SQLite is embedded, no separate database server is required.

Development Workflow

1. Create a Git repository before writing code.
2. Develop in small, testable modules.

3. Commit after every stable milestone.
4. Maintain documentation alongside the code.
5. Use automated formatting and linting tools.

Long-Term Expandability

- Networked displays
- Multiple operator consoles
- Sound notifications
- Automatic event logging
- Competition replay
- Remote monitoring

Recommended Next Steps

1. Complete the functional specification.
2. Create a Software Requirements Specification (SRS).
3. Design the state machine.
4. Design the user interface.
5. Create the Python project.
6. Implement the state engine.
7. Implement timers.
8. Implement the display renderer.
9. Implement the operator interface.
10. Perform integration and field testing.

Final Recommendation

The Master Flightline Clock should be engineered as a modular real-time application centred on a deterministic state machine. Separating the timer engine, business logic, operator interface and display renderer from the outset will produce a professional-grade application that is reliable, maintainable and straightforward to extend over time.